

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 1

DATABASE CREATION AND ER MODELING

1. Aim

To design an Entity-Relationship (ER) diagram for a real-world case study, map it to a relational schema and create the corresponding database and tables in MySQL.

2. Problem Statement

An engineering college wants to replace its paper registers with a database for managing academic records. The requirements collected for this case study are:

- The college has several **departments**. Each department has an id, a unique department code, a name, a building and an annual budget.
- Every **student** belongs to exactly one department. For a student the college stores an id, a unique roll number, name, gender, date of birth, city, e-mail and one or more phone numbers. The age of a student should be obtainable from the date of birth.
- Every **instructor** works in one department and has an id, name, designation, salary and e-mail. A senior instructor may act as mentor for several other instructors.
- A department **offers** many **courses**; each course has an id, title and credits, is taught by one instructor and may have one prerequisite course.
- A student enrolls in many courses and a course has many students. For every enrollment the semester, marks obtained and grade are recorded.

Identify the entities, attributes, relationships and cardinalities, draw the ER diagram, map it to a relational schema and create the database *college_db* in MySQL.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To understand the basic concepts of the ER model: entities, attributes, relationships and keys.
2. To identify cardinality ratios and participation constraints from requirements.
3. To draw an ER diagram for a real-world case study using Chen notation.
4. To map the ER diagram to a relational schema using standard mapping rules.
5. To create a database and its tables in MySQL and verify their structure.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files
5	Diagram tool (draw.io) or pencil and paper	Drawing the ER diagram

5. Theory

5.1 Database and Database Design

A **database** is an organised collection of logically related data, and a **Database Management System (DBMS)** is the software that stores, retrieves and protects that data. Database design is normally carried out in three stages: **conceptual design** (an ER diagram that is independent of any DBMS), **logical design** (conversion of the ER diagram into relations/tables) and **physical design** (data types, indexes and storage in a particular DBMS such as MySQL).

5.2 Entity-Relationship (ER) Model

The ER model, proposed by P. P. Chen (1976), describes the real world as a set of **entities**, their **attributes** and the **relationships** among them.

- **Entity / Entity set:** a real-world object that can be distinctly identified (e.g. one student) and the collection of all similar objects (STUDENT).
- **Strong and weak entity:** a strong entity has its own primary key; a weak entity depends on an owner entity for identification and is drawn with a double rectangle.
- **Simple vs composite attribute:** *gender* cannot be divided further; an address made of street, city and pin code is composite.
- **Single-valued vs multivalued attribute:** a student has one *dob* but may have several *phone* numbers (double ellipse).
- **Stored vs derived attribute:** *age* can be derived from the stored attribute *dob* (dashed ellipse).
- **Key attribute:** an attribute whose value uniquely identifies an entity (underlined), e.g. *student_id*.
- **Relationship:** an association among entities, e.g. STUDENT *enrolls* in COURSE. Relationships may have their own attributes (marks, grade).
- **Degree:** number of participating entity sets - unary (recursive), binary or ternary. MENTORS (instructor mentors instructor) is a recursive relationship.

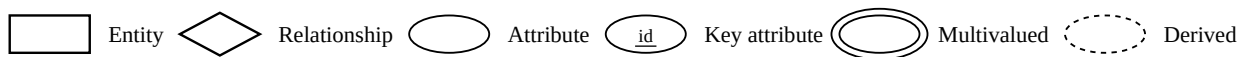
5.3 Cardinality Ratio and Participation

The **cardinality ratio** specifies the maximum number of relationship instances in which an entity can take part:

Ratio	Meaning	Example from the case study
1 : 1	One entity of A is related to at most one entity of B and vice versa	A department has one head (HOD) instructor
1 : N	One entity of A is related to many entities of B, each B to one A	One department offers many courses
M : N	Many entities of A are related to many entities of B	Students enroll in many courses; a course has many students

Participation is *total* when every entity must take part in the relationship (every student must belong to a department; drawn as a double line) and *partial* otherwise (not every instructor mentors another instructor).

5.4 Chen Notation Used



5.5 Mapping an ER Diagram to a Relational Schema

1. **Strong entity:** create a relation with all simple attributes; the key attribute becomes the PRIMARY KEY. Composite attributes are stored as their components; derived attributes are not stored.
2. **Weak entity:** create a relation whose key is (owner key + partial key), with a foreign key to the owner.
3. **Binary 1:1 relationship:** add the primary key of one side as a (unique) foreign key in the other, preferably on the side with total participation.
4. **Binary 1:N relationship:** add the primary key of the '1' side as a foreign key in the relation of the 'N' side.
5. **Binary M:N relationship:** create a new relation containing the primary keys of both sides (together forming its primary key) and the attributes of the relationship.
6. **Multivalued attribute:** create a new relation (owner key, attribute) with both columns as the primary key.
7. **Recursive relationship:** add a foreign key that references the primary key of the same relation (e.g. mentor_id references instructor).

5.6 MySQL Commands Used

Command	Purpose
CREATE DATABASE db;	Creates a new database (schema)
USE db;	Makes the database the default for later statements
CREATE TABLE t (...);	Creates a relation with columns, data types and keys
SHOW TABLES; DESCRIBE t;	Lists tables / shows the structure of a table
information_schema	System database that stores metadata such as keys and constraints

6. Procedure / Algorithm

1. Read the requirements of the case study and underline the nouns; nouns that have independent existence become **entities** and descriptive nouns become **attributes**.
2. Underline the verbs that associate entities; these become **relationships**.
3. For every relationship decide the cardinality ratio (1:1, 1:N, M:N) and participation.
4. Choose a key attribute for every entity and classify the remaining attributes (composite, multivalued, derived).
5. Draw the ER diagram using Chen notation (Figure 1, Section 9).
6. Apply the mapping rules of Section 5.5 to obtain the relational schema.
7. Start the MySQL server and log in with `mysql -u root -p`.
8. Create the database and the tables in dependency order: parent tables (department) first, then tables that refer to them.
9. Verify the result using SHOW TABLES, DESCRIBE and information_schema, insert one sample row per table and record the output.

6.1 Entities and Attributes Identified

Entity	Key attribute	Other attributes
DEPARTMENT	dept_id	dept_code (unique), dept_name, building, budget
STUDENT	student_id	roll_no (unique), student_name, gender, dob, city, email, phone (multivalued), age (derived from dob)
INSTRUCTOR	instr_id	instr_name, designation, salary, email
COURSE	course_id	title, credits

6.2 Relationships and Cardinalities Identified

Relationship	Entities	Cardinality	Participation / attributes
BELONGS_TO	STUDENT - DEPARTMENT	N : 1	Total for STUDENT
WORKS_IN	INSTRUCTOR - DEPARTMENT	N : 1	Partial
OFFERS	DEPARTMENT - COURSE	1 : N	Total for COURSE
TEACHES	INSTRUCTOR - COURSE	1 : N	Partial (a course may be unassigned)
ENROLLS	STUDENT - COURSE	M : N	Attributes: semester, marks, grade
MENTORS	INSTRUCTOR - INSTRUCTOR	1 : N (recursive)	Roles: mentor, mentee
PREREQ_OF	COURSE - COURSE	1 : N (recursive)	Roles: prerequisite, course

7. SQL Queries / Program

Q1: Create the database and select it

```
DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
SHOW DATABASES LIKE 'college%';
USE college_db;
```

Q2: Strong entities DEPARTMENT and INSTRUCTOR (with WORKS_IN and MENTORS)

```
CREATE TABLE department (
    dept_id    INT          PRIMARY KEY,
    dept_code  VARCHAR(10)  NOT NULL UNIQUE,
    dept_name  VARCHAR(40)  NOT NULL,
    building   VARCHAR(20),
    budget     DECIMAL(12,2)
);

CREATE TABLE instructor (
    instr_id   INT          PRIMARY KEY,
    instr_name VARCHAR(40)   NOT NULL,
    designation VARCHAR(20),
    salary      DECIMAL(10,2),
    email       VARCHAR(40),
    dept_id     INT,         -- WORKS_IN (N:1) -> FK on the N side
    mentor_id   INT,         -- MENTORS (recursive 1:N)
    FOREIGN KEY (dept_id) REFERENCES department (dept_id),
    FOREIGN KEY (mentor_id) REFERENCES instructor (instr_id)
);
```

Q3: Entity STUDENT (with BELONGS_TO) and multivalued attribute phone

```

CREATE TABLE student (
    student_id    INT          PRIMARY KEY,
    roll_no       VARCHAR(10)  NOT NULL UNIQUE,
    student_name  VARCHAR(40)  NOT NULL,
    gender        CHAR(1),
    dob           DATE,         -- age is derived from dob, so it is not stored
    city          VARCHAR(30),
    email         VARCHAR(40),
    dept_id       INT NOT NULL, -- BELONGS_TO (N:1)
    FOREIGN KEY (dept_id) REFERENCES department (dept_id)
);

CREATE TABLE student_phone ( -- multivalued attribute
    student_id INT,
    phone       CHAR(10),
    PRIMARY KEY (student_id, phone),
    FOREIGN KEY (student_id) REFERENCES student (student_id)
);

```

Q4: Entity COURSE (OFFERS, TEACHES, PREREQ_OF) and M:N relationship ENROLLS

```

CREATE TABLE course (
    course_id CHAR(5)    PRIMARY KEY,
    title      VARCHAR(40) NOT NULL,
    credits    TINYINT,
    dept_id    INT NOT NULL, -- OFFERS (1:N)
    instr_id   INT,         -- TEACHES (1:N)
    prereq_id  CHAR(5),     -- PREREQ_OF (recursive 1:N)
    FOREIGN KEY (dept_id) REFERENCES department (dept_id),
    FOREIGN KEY (instr_id) REFERENCES instructor (instr_id),
    FOREIGN KEY (prereq_id) REFERENCES course (course_id)
);

CREATE TABLE enrollment ( -- ENROLLS (M:N) becomes a separate relation
    student_id INT,
    course_id   CHAR(5),
    semester    TINYINT,
    marks        INT,
    grade        CHAR(2),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student (student_id),
    FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

Q5: Verify the created tables and their structure

```

SHOW TABLES;
DESCRIBE student;
DESCRIBE enrollment;

```

Q6: List every foreign key, i.e. every relationship of the ER diagram

```

SELECT table_name      AS child_table,
       column_name     AS fk_column,
       referenced_table_name AS parent_table,
       referenced_column_name AS parent_column
FROM   information_schema.key_column_usage
WHERE  table_schema = 'college_db'
       AND referenced_table_name IS NOT NULL
ORDER BY child_table, fk_column;

```

Q7: Store one sample tuple per entity and compute the derived attribute age

```

INSERT INTO department VALUES (10, 'CSE', 'Computer Engineering', 'A Block', 2500000);
INSERT INTO instructor VALUES (101, 'Anil Kulkarni', 'Professor', 185000,
                                'anil.k@example.com', 10, NULL);
INSERT INTO student VALUES (1, 'S24CS001', 'Aarav Sharma', 'M', '2005-03-14',
                              'Pune', 'aarav@example.com', 10);
INSERT INTO student_phone VALUES (1, '9876500011'), (1, '9876500012');
INSERT INTO course VALUES ('CS301', 'Database Management Systems', 3, 10, 101, NULL);
INSERT INTO enrollment VALUES (1, 'CS301', 4, 91, '0');
SELECT s.roll_no, s.student_name, s.dob,
       TIMESTAMPDIFF(YEAR, s.dob, '2026-10-09') AS age_on_2026_10_09,
       COUNT(p.phone) AS phone_count
FROM   student s LEFT JOIN student_phone p ON p.student_id = s.student_id
GROUP BY s.student_id;

```

The date is written as a constant so that the result is reproducible; in practice `CURDATE()` is used to compute the current age.

8. Sample Input / Database Tables

The input to this practical is the set of requirements given in Section 2. The tables created from the relational schema are summarised below as a data dictionary, read from `information_schema.columns` after executing Q1-Q4.

Data dictionary of college_db

Table	Column	Data type	Null	Key
department	dept_id	int	NO	PRI
department	dept_code	varchar(10)	NO	UNI
department	dept_name	varchar(40)	NO	
department	building	varchar(20)	YES	
department	budget	decimal(12,2)	YES	
instructor	instr_id	int	NO	PRI
instructor	instr_name	varchar(40)	NO	
instructor	designation	varchar(20)	YES	
instructor	salary	decimal(10,2)	YES	
instructor	email	varchar(40)	YES	
instructor	dept_id	int	YES	MUL
instructor	mentor_id	int	YES	MUL
student	student_id	int	NO	PRI
student	roll_no	varchar(10)	NO	UNI
student	student_name	varchar(40)	NO	
student	gender	char(1)	YES	
student	dob	date	YES	
student	city	varchar(30)	YES	
student	email	varchar(40)	YES	
student	dept_id	int	NO	MUL
student_phone	student_id	int	NO	PRI

Table	Column	Data type	Null	Key
student_phone	phone	char(10)	NO	PRI
course	course_id	char(5)	NO	PRI
course	title	varchar(40)	NO	
course	credits	tinyint	YES	
course	dept_id	int	NO	MUL
course	instr_id	int	YES	MUL
course	prereq_id	char(5)	YES	MUL
enrollment	student_id	int	NO	PRI
enrollment	course_id	char(5)	NO	PRI
enrollment	semester	tinyint	YES	
enrollment	marks	int	YES	
enrollment	grade	char(2)	YES	

Key column legend: PRI = primary key, UNI = unique key, MUL = non-unique index (created automatically by MySQL for every foreign key column).

9. Output

9.1 ER Diagram (Chen Notation)

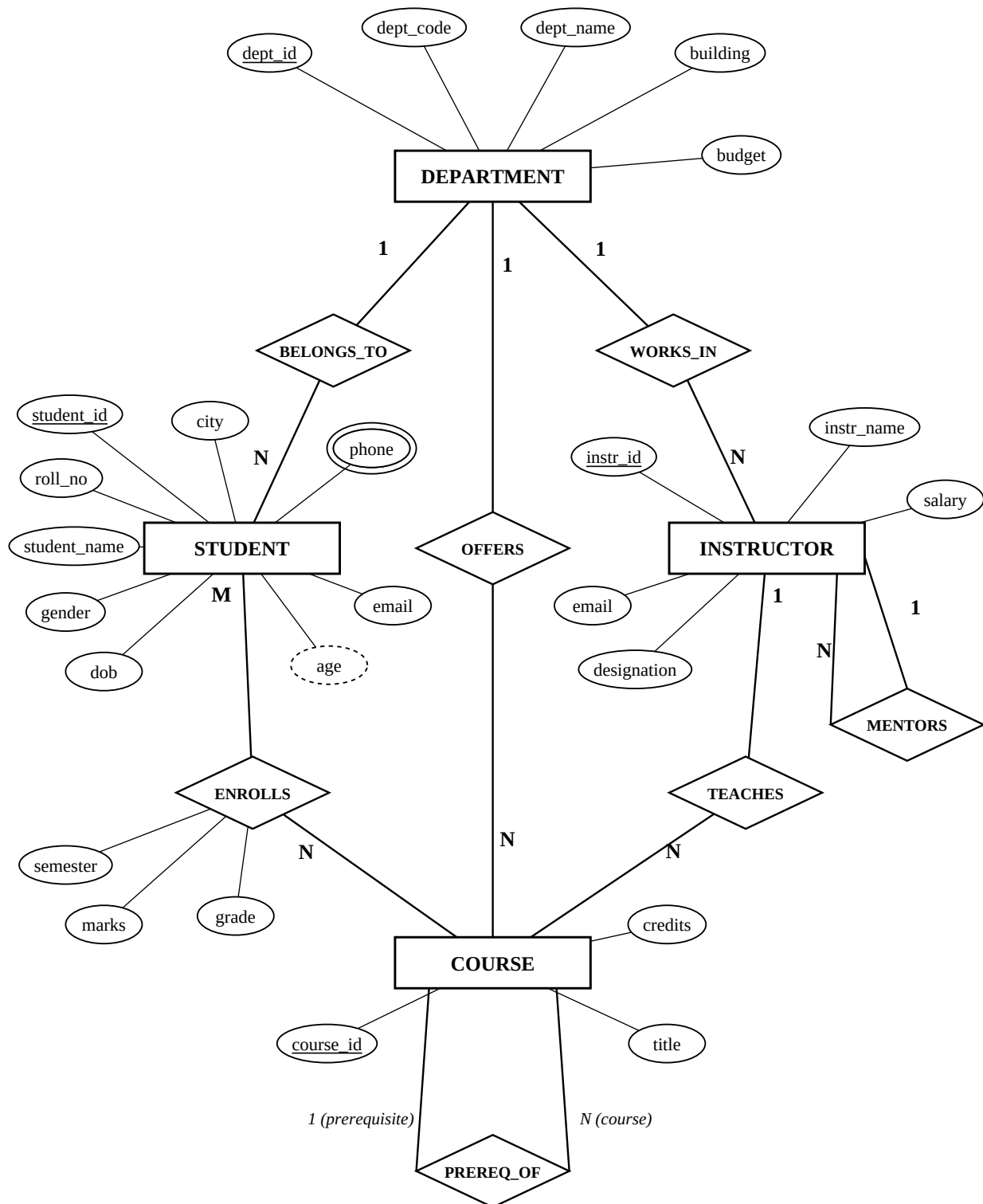


Figure 1: ER diagram of the College Academic Management System

9.2 Relational Schema Obtained by Mapping

Primary keys are underlined and foreign keys are marked with an asterisk (*).

Relation	Schema	Mapping rule applied
DEPARTMENT	DEPARTMENT(<u>dept_id</u> , dept_code, dept_name, building, budget)	Strong entity
INSTRUCTOR	INSTRUCTOR(<u>instr_id</u> , instr_name, designation, salary, email, dept_id*, mentor_id*)	Strong entity; WORKS_IN (N:1) and recursive MENTORS (1:N) as foreign keys
STUDENT	STUDENT(<u>student_id</u> , roll_no, student_name, gender, dob, city, email, dept_id*)	Strong entity; BELONGS_TO (N:1); age not stored
STUDENT_PHONE	STUDENT_PHONE(<u>student_id</u> *, <u>phone</u>)	Multivalued attribute
COURSE	COURSE(<u>course_id</u> , title, credits, dept_id*, instr_id*, prereq_id*)	Strong entity; OFFERS, TEACHES (1:N) and recursive PREREQ_OF
ENROLLMENT	ENROLLMENT(<u>student_id</u> *, <u>course_id</u> *, semester, marks, grade)	M:N relationship ENROLLS

9.3 Output of the SQL Statements

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Create the database and select it

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.000 sec)

mysql> SHOW DATABASES LIKE 'college%';
+-----+
| Database (college%) |
+-----+
| college_db          |
+-----+
1 row in set (0.00 sec)
```

Output of Q2: Strong entities DEPARTMENT and INSTRUCTOR (with WORKS_IN and MENTORS)

```
mysql> CREATE TABLE department (
->   dept_id      INT           PRIMARY KEY,
->   dept_code   VARCHAR(10)   NOT NULL UNIQUE,
->   dept_name   VARCHAR(40)   NOT NULL,
->   building    VARCHAR(20),
->   budget      DECIMAL(12,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE instructor (
->   instr_id     INT           PRIMARY KEY,
->   instr_name   VARCHAR(40)   NOT NULL,
->   designation  VARCHAR(20),
->   ...
-> );
Query OK, 0 rows affected (0.01 sec)
```

Output of Q3: Entity STUDENT (with BELONGS_TO) and multivalued attribute phone

```
mysql> CREATE TABLE student (
->   student_id   INT           PRIMARY KEY,
->   roll_no      VARCHAR(10)   NOT NULL UNIQUE,
->   student_name VARCHAR(40)   NOT NULL,
->   ...
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student_phone (
->   student_id   INT,
->   phone        CHAR(10),
->   PRIMARY KEY (student_id, phone),
->   FOREIGN KEY (student_id) REFERENCES student (student_id)
-> );
Query OK, 0 rows affected (0.01 sec)
```

Output of Q4: Entity COURSE (OFFERS, TEACHES, PREREQ_OF) and M:N relationship ENROLLS

```
mysql> CREATE TABLE course (
->   course_id   CHAR(5)       PRIMARY KEY,
->   title       VARCHAR(40)   NOT NULL,
->   credits     TINYINT,
->   ...
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE enrollment (
->   student_id  INT,
->   course_id   CHAR(5),
->   semester    TINYINT,
->   ...
-> );
Query OK, 0 rows affected (0.02 sec)
```

Output of Q5: Verify the created tables and their structure

```
mysql> SHOW TABLES;
+-----+
| Tables_in_college_db |
+-----+
| course                |
| department            |
| enrollment            |
| instructor            |
| student              |
```

```
| student_phone |
+-----+
6 rows in set (0.00 sec)

mysql> DESCRIBE student;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| student_id | int           | NO   | PRI | NULL    |       |
| roll_no    | varchar(10)   | NO   | UNI | NULL    |       |
| student_name | varchar(40)   | NO   |     | NULL    |       |
| gender     | char(1)       | YES  |     | NULL    |       |
| dob        | date          | YES  |     | NULL    |       |
| city       | varchar(30)   | YES  |     | NULL    |       |
| email      | varchar(40)   | YES  |     | NULL    |       |
| dept_id    | int           | NO   | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> DESCRIBE enrollment;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| student_id | int           | NO   | PRI | NULL    |       |
| course_id  | char(5)       | NO   | PRI | NULL    |       |
| semester   | tinyint       | YES  |     | NULL    |       |
| marks      | int           | YES  |     | NULL    |       |
| grade      | char(2)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Output of Q6: List every foreign key, i.e. every relationship of the ER diagram

```
mysql> SELECT table_name AS child_table,
-> column_name AS fk_column,
-> referenced_table_name AS parent_table,
-> referenced_column_name AS parent_column
-> FROM information_schema.key_column_usage
-> WHERE table_schema = 'college_db'
-> AND referenced_table_name IS NOT NULL
-> ORDER BY child_table, fk_column;
+-----+-----+-----+-----+
| child_table | fk_column | parent_table | parent_column |
+-----+-----+-----+-----+
| course      | dept_id  | department  | dept_id       |
| course      | instr_id | instructor   | instr_id      |
| course      | prereq_id | course      | course_id     |
| enrollment  | course_id | course      | course_id     |
| enrollment  | student_id | student     | student_id    |
| instructor  | dept_id  | department  | dept_id       |
| instructor  | mentor_id | instructor   | instr_id      |
| student     | dept_id  | department  | dept_id       |
| student_phone | student_id | student     | student_id    |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

Output of Q7: Store one sample tuple per entity and compute the derived attribute age

```
mysql> INSERT INTO department VALUES (10, 'CSE', 'Computer Engineering', 'A Block', 2500000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO instructor VALUES (101, 'Anil Kulkarni', 'Professor', 185000,
-> 'anil.k@example.com', 10, NULL);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student VALUES (1, 'S24CS001', 'Aarav Sharma', 'M', '2005-03-14',
-> 'Pune', 'aarav@example.com', 10);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student_phone VALUES (1, '9876500011'), (1, '9876500012');
Query OK, 2 rows affected (0.00 sec)
```

```

Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO course VALUES ('CS301', 'Database Management Systems', 3, 10, 101, NULL);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO enrollment VALUES (1, 'CS301', 4, 91, '0');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT s.roll_no, s.student_name, s.dob,
->         TIMESTAMPDIFF(YEAR, s.dob, '2026-10-09') AS age_on_2026_10_09,
->         COUNT(p.phone) AS phone_count
-> FROM    student s LEFT JOIN student_phone p ON p.student_id = s.student_id
-> GROUP BY s.student_id;
+-----+-----+-----+-----+-----+
| roll_no | student_name | dob       | age_on_2026_10_09 | phone_count |
+-----+-----+-----+-----+-----+
| S24CS001 | Aarav Sharma | 2005-03-14 | 21 | 2 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

10. Result

The ER diagram of the College Academic Management System was designed with four entities (DEPARTMENT, STUDENT, INSTRUCTOR, COURSE), seven relationships including two recursive relationships, one M:N relationship, one multivalued and one derived attribute. It was mapped to six relations and the database *college_db* with all six tables, primary keys and nine foreign keys was successfully created and verified in MySQL 8.0.

11. Conclusion

ER modeling helps to capture the requirements of a real-world system before any table is created. Each element of the ER diagram maps to the relational model in a systematic way: entities become tables, 1:N relationships become foreign keys on the N side, the M:N relationship becomes a separate table with a composite primary key, the multivalued attribute becomes a separate table and the derived attribute is computed at query time. The schema created here is refined with additional constraints in Practical 2.